FoodieGo Website Documentation

FoodieGo is a Zomato-style food delivery web application where users can browse restaurants, view menus, create an account, add food to a cart, and place cash-on-delivery orders.

Project type: MERN full-stack web application

1. Website Overview

The website has a customer-facing ordering flow. Users can search restaurants by restaurant name, cuisine, or location, including Pune locations such as Kothrud, Koregaon Park, and Hinjewadi. A logged-in user can add menu items to the cart, enter delivery details, and place an order without an online payment gateway.

User Features

- Signup and login using JWT authentication.
- Browse restaurant cards with rating, location, and delivery time.
- Search by location, cuisine, or restaurant name.
- View menu items by restaurant and category.
- Add, update, and remove cart items.
- Place cash-on-delivery orders.
- View order history and order status.

Admin/API Features

- Admin-protected restaurant creation endpoint.
- Admin-protected menu item creation endpoint.
- Admin order listing endpoint.
- Admin order status update endpoint.
- Seed script for sample restaurants and menu data.

2. How The Website Works

```
User opens FoodieGo
  -> Restaurant list loads from Express API
  -> User searches by food, cuisine, or location
  -> User opens a restaurant menu
  -> User logs in or signs up
  -> User adds menu items to cart
  -> Cart is saved in MongoDB
  -> User enters delivery address
  -> User places COD order
  -> Backend creates order and clears cart
  -> User sees the order in My Orders
```

Authentication Flow

1. User submits signup or login form from the React frontend.
2. Express validates the request.
3. Passwords are hashed using `bcryptjs`.
4. Successful login/register returns a JWT token.
5. The frontend stores the token in `localStorage`.
6. Protected requests send the token in the `Authorization` header.

Ordering Flow

1. User selects a restaurant and menu item.
2. The cart API checks that all cart items are from one restaurant.
3. The cart stores menu item IDs and quantities for the logged-in user.
4. At checkout, the order API calculates subtotal, delivery fee, and total.
5. The order is saved with `paymentMethod: COD` and `paymentStatus: pending`.

3. Tech Stack Used

Layer	Technology	Purpose
Frontend	React + Vite	Builds the single-page web app and development server.

Layer	Technology	Purpose
Styling	Tailwind CSS	Creates responsive layouts and reusable utility-based styles.
Routing	React Router	Handles frontend pages like home, auth, cart, and orders.
State	Zustand	Stores authentication and cart state on the frontend.
HTTP Client	Axios	Sends API requests from React to Express.
Backend	Node.js + Express	Provides REST APIs for auth, restaurants, carts, and orders.
Database	MongoDB + Mongoose	Stores users, restaurants, menu items, carts, and orders.
Authentication	JWT + bcryptjs	Protects private routes and securely stores passwords.

4. Architecture

```
React Frontend (client)
  Pages: Home, Restaurant, Auth, Cart, Orders
  State: authStore, cartStore
  API Client: Axios

        |
        | HTTP JSON requests
        v

Express Backend (server)
  Routes: auth, restaurants, cart, orders
  Middleware: JWT protect, adminOnly, error handling
  Models: User, Restaurant, MenuItem, Cart, Order
```

```
        |
        | Mongoose ODM
        v

MongoDB Database
  Collections: users, restaurants, menuitems, carts, orders
```

Folder Structure

```
food-delivery/
  client/
    src/
      components/
      pages/
      store/
      lib/
  server/
    src/
      routes/
      models/
      middleware/
      lib/
      seed.js
```

5. Main Database Models

- **User:** name, email, password, role, addresses.
- **Restaurant:** name, description, image, cuisines, rating, location, delivery fee.
- **MenuItem:** restaurant, name, description, price, category, veg/non-veg, availability.
- **Cart:** user, restaurant, menu items, quantities.
- **Order:** user, restaurant, items, delivery address, subtotal, delivery fee, total, COD status.

6. Current Limitations And Next Improvements

- Payment gateway is intentionally not included yet.

- Restaurant and menu image upload can later be added using Cloudinary.
- Admin dashboard UI can be added on top of the existing admin APIs.
- Live delivery tracking can be added later with order status updates or sockets.
- Location search can later use real maps and geolocation APIs.